

CRYPTOGRAPHER

A PROJECT FILE BY
Shreyansh Mishra
26BCE10245

September 27, 2026

—

Python Essentials

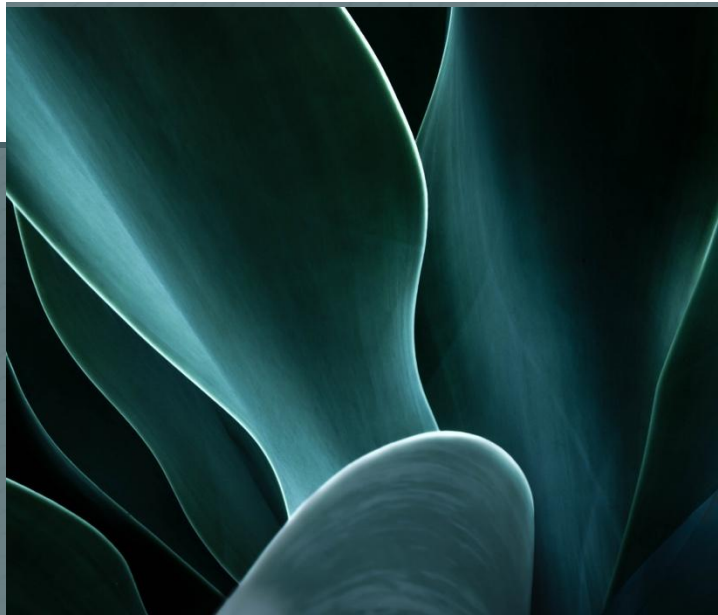
—

Dr. Lakshmi D.
Dr. Poonkuntran S.

ACKNOWLEDGMENT

I would like to thank Dr. Lakshmi D. and Dr. Poonkuntran S. for their immense great teaching and making the concepts of this particular course of **Essentials of Python** clear as a crystal.

Github Repository:



INTRODUCTION

The **Simple Cryptographer** is a python program which provides us with the ability to readily performing encryption and decryption of ciphers and even stack ciphers. The application consists of 10 classical ciphers including **Caesar**, **A1Z26**, **Atbash**, **Baconian**, **Beaufort**, **Vigenère**, **XOR**, **Scytale**, **ROT13**, and **Morse-Code**.

To cater different user environments, the project is deployed with two distinct interfaces, a robust command line interface (**CLI**) and another polished object-oriented graphical user interface (**GUI**) which is developed via Tkinter library.

The application emphasizes usability, efficiency and modular design.

AGENDA

Problem Statement	4
Requirements	5
System Architecture	6
Design Visualization	7
Complexities	8
Design Decisions	10
Implementation Details	11
Challenges Faced	12
Mockups	13
Conclusion	14

PROBLEM STATEMENT

AIM

In the realm of cryptography education and hobbyist code-breaking, individuals often rely on fragmented, ad-laden, or online-only web tools to explore and manipulate historical ciphers. There is a notable lack of cohesive, privacy-respecting, and offline-capable applications that consolidate multiple classic encryption methods into a single, user-friendly interface. Furthermore, students studying computer science or introductory cryptography often struggle to find clear, accessible, and object-oriented implementations of these algorithms to study, experiment with, and execute locally.

SCOPE OF PROJECT

The scope of Simple Cryptographer encompasses the design, development and deployment of a local python application that performs bidirectional text translations for 10 classical ciphers. The project includes two types of versions, where one consists of a command-line-interface design and other offers a fully functional graphical user interface.

The project is strictly limited to classic, text-based cryptographic algorithms for most of the ciphers, operating on the time complexity $O(n)$. The scope excludes modern encryption standards such as:

1. *AES*
2. *RSA*
3. *SHA*
4. *Network based communication*
5. *File-level-data encryption or sharing features.*

Final deliverable includes a .exe file ensuring frictionless deployment for end-users without requiring pre-configured Python IDE or environments.



REQUIREMENTS

FUNCTIONAL REQUIREMENTS

The system must qualify for the following criteria:

- **Algorithm Support**
Implement a translation layer for encoding and decoding Caesar, A1Z26, Atbash, Baconian, Beaufort, Vigenère, Xor, Scytale, Rot13, Morse Code
- **Dynamic Interface**
The system must be dynamically prompt for CLI or GUI specific parameters (key, shift, diameter) based strictly on the selected cipher algorithm
- **Bidirectional Algorithm**
User must be able to toggle between encryption and decryption
- **Clipboard Management**
The GUI must allow users to copy the output text directly to the system clipboard and paste from it
- **Error Handling**
The system might react stably if prompted with invalid inputs and display graceful error messages instead of crashing

NON - FUNCTIONAL REQUIREMENTS

The system must qualify the following criteria

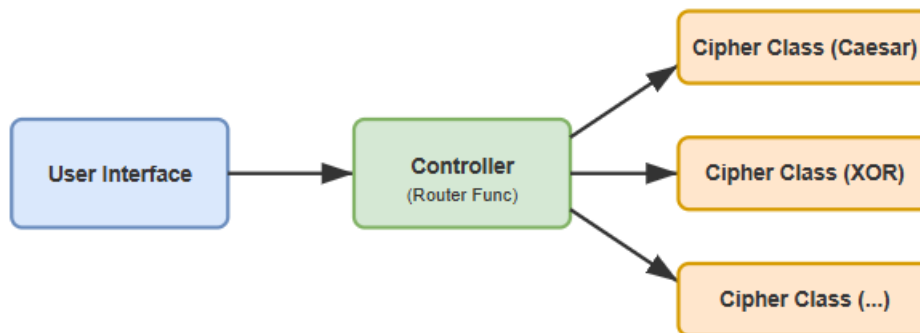
- **Performance**
Time complexity for translations remains generally near $O(n)$ where n is the length of the string, ensuring instant processing even for larger text blocks
- **Zero Dependency**
The application must run on any standard python 3.x installation without requiring any extra dependencies of libraries
- **Modularity and Future Proof**
Through OOPS, adding more ciphers should be easy as biting a cake since we've separated every small part of the program so that modularity can be maintained.
- **Nightly GUI**
The GUI features a modern dark nightly aesthetic to reduce eyestrain

SYSTEM ARCHITECTURE

SYSTEM ARCHITECTURE

The application follows a two-tier architecture dividing the presentation and business logic into multiple parts.

- **Presentation Layer (UI)**
Managed by the `CipherNightGUI` class. It goes hand in hand with Tkinter. And contributes for the styling of whole page via `ttk.Style` and binds event listeners for other styling components such as buttons and input boxes.
- **Business Logic Layer (Engine)**
Composed of modular translation functions. `process_caesar_text`, `process_vigenere_text`, `process_xor_text` all fall in this classification.
- **Routing / Controller**
The `process_translation` function acts as a centralized router, taking the selected cipher and delegating the input text to the appropriate cryptographic engine.



COMPLEXITIES

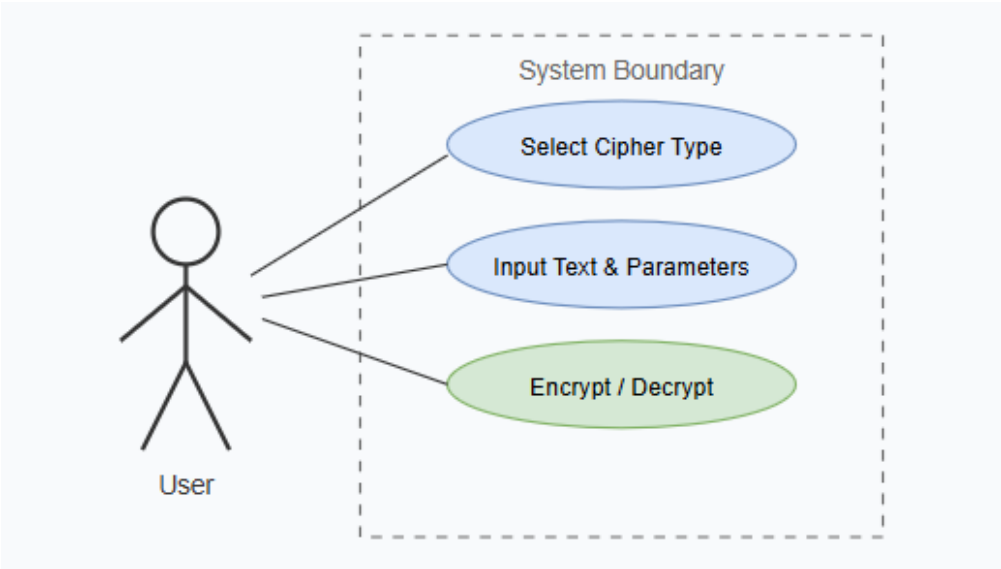
CIPHER COMPLEXITIES

Cipher	Time	Space	Parameters Involved
Caesar	$O(n)$	$O(n)$	Text length (n)
A1Z26	$O(n)$	$O(n)$	Text length (n)
Atbash	$O(n)$	$O(n)$	Text length (n)
Baconian	$O(n)$	$O(n)$	Text length (n)
Beaufort	$O(n)$	$O(n)$	Text length (n), Key length (m)
Vigenère	$O(n)$	$O(n)$	Text length (n), Key length (m)
XOR	$O(n)$	$O(n)$	Text length (n), Key length (m)
Scytale	$O(n + d)$	$O(n + d)$	Text length (n), Diameter (d)
ROT13	$O(n)$	$O(n)$	Text length (n)
Morse Code	$O(n)$	$O(n)$	Text length (n)

DESIGN DIAGRAMS

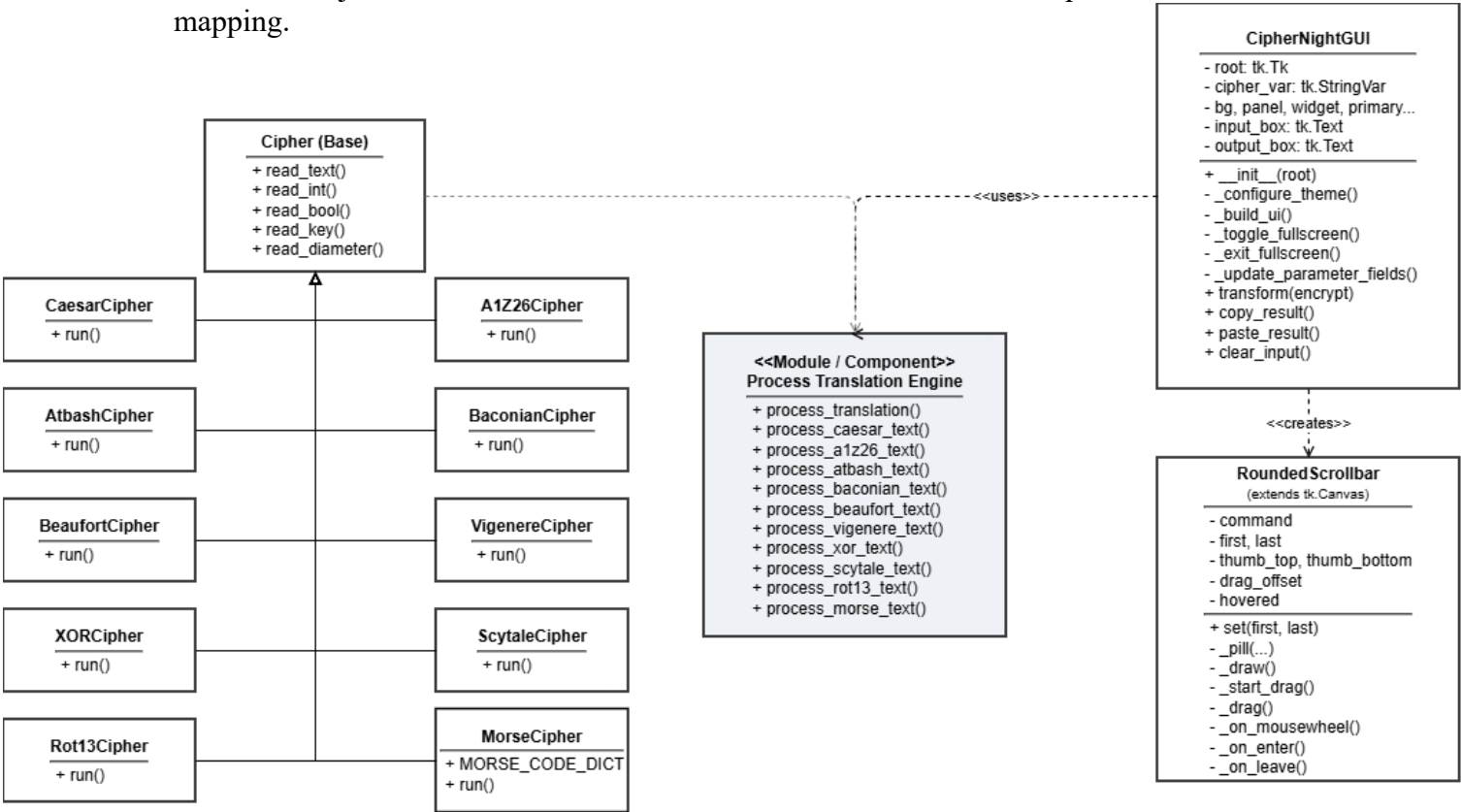
USE CASE DIAGRAM

This diagram represents the interactions between the user and the application.



CLASS COMPONENT DIAGRAM

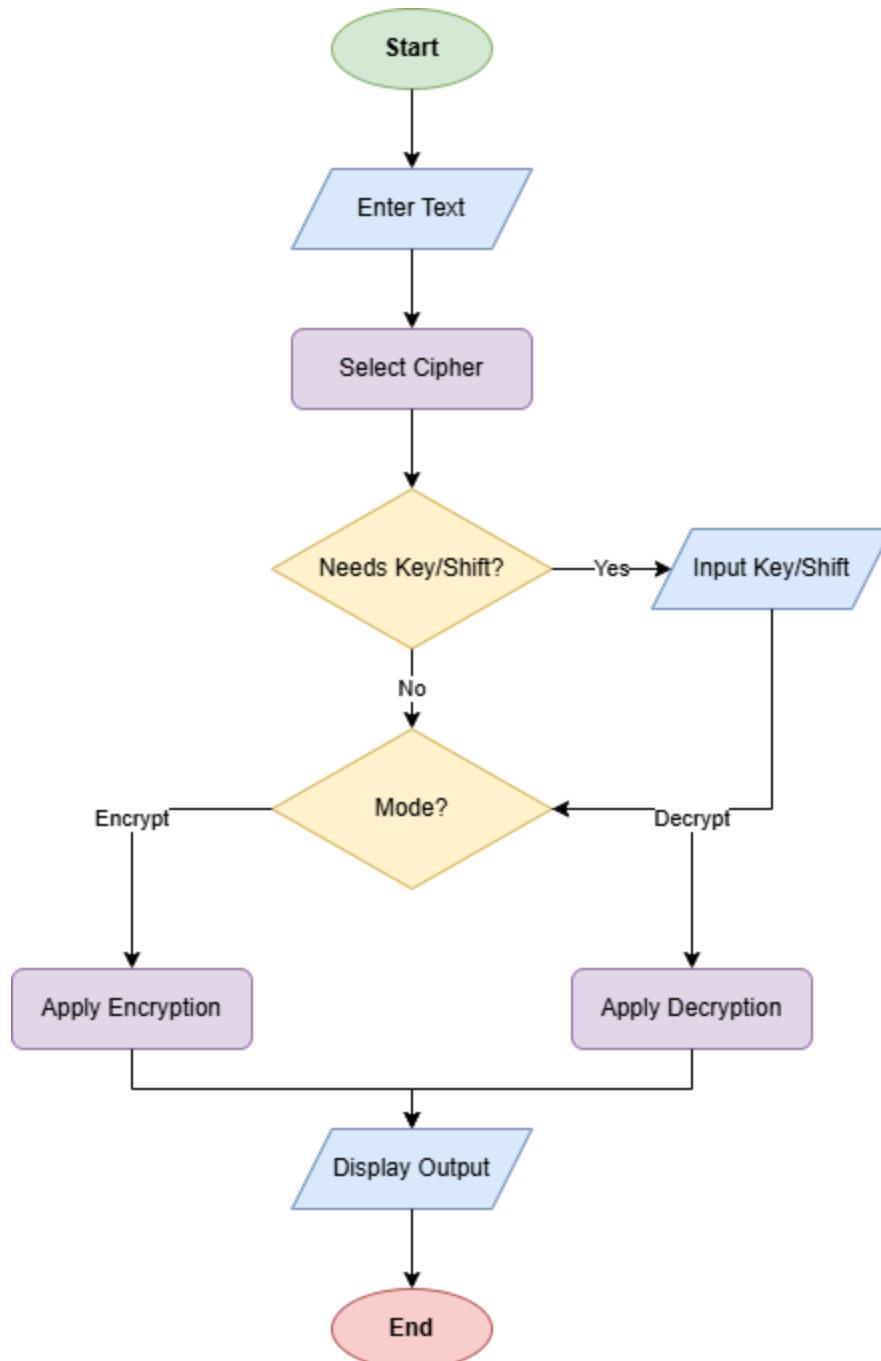
Shows the object-oriented inheritance structure used for the terminal implementation mapping.



DESIGN DIAGRAMS

WORKFLOW DIAGRAM / SEQUENCE DIAGRAM

This flowchart represents the workflow of the whole program that can be interpreted as the former algorithm for the whole program. This flowchart has been made with a high level of abstraction.



DESIGN DECISIONS

DECISIONS TAKEN

1. Tkinter for GUI

Tkinter was chosen due to its vast compatibility and easy to understand code, like html + css.

2. Dictionary Lookups

For Atbash, Morse Code and Baconian, predefined dictionaries were used instead of calculating the shift of each character such that algorithms can achieve $O(1)$ constant time lookups and during string parsing.

3. Event Driven UI Updates

The `'combobox'` utilizes the `<combo_selected>` virtual event to dynamically hide and show the inputs for shift, key, and diameter as per the selected cipher, preventing user confusion by reducing redundant inputs.

4. Object Oriented Programming

The whole program is divided into many objects so that adding another cipher or another input method or just debugging the code is simple. Even the main components of the app that drive the logic of the ciphers are **encapsulated**.

WHY ARE THERE TWO APPS FOR THE SAME PROGRAM?

I focused on developing a single command line script, typically a terminal based application at first, but realized that every user can't adapt to a terminal style of giving commands. I got to this conclusion after I gave my application file for testing to my friends, who are of different domains.

With this vision, after the completion of the CLI version of the program, I found about 'Tkinter' and with the help of Luna I started building the graphical interface for the program and added the OOPS method so that I can simultaneously implement it and make the whole process smooth.

IMPLEMENTATION DETAILS

ALGORITHM'S FLOWCHART

The python applications employs strict type conversions. For instance, ciphers like Vigenere dynamically convert characters to their Unicode integer values ``ord()``. Apply the mathematical shift bounded by module 26, and map your way back using ``chr()``.

```
Translation

def process_translation(cipher_name, text, encrypt=True, key="", shift=0, diameter=1):
    # For outputs, we've made a whole translation function for better readability
    name = cipher_name.strip().lower()

    if "caesar" in name: return process_caesar_text(text, shift, encrypt)
    if "alz26" in name or "alz" in name: return process_alz26_text(text, encrypt)
    if "atbash" in name: return process_atbash_text(text)
    if "baconian" in name: return process_baconian_text(text, encrypt)
    if "beaufort" in name: return process_beaufort_text(text, key, encrypt)
    if "vigenere" in name: return process_vigenere_text(text, key, encrypt)
    if "xor" in name: return process_xor_text(text, key, encrypt)
    if "scytale" in name: return process_scytale_text(text, diameter, encrypt)
    if "rot13" in name: return process_rot13_text(text)
    if "morse" in name: return process_morse_text(text, encrypt)

    raise ValueError(f"Unable to trace the cipher: {cipher_name}")
```

CONCLUSION

The GUI captures input via a ``tk.Text`` widget, sanitizes it by stripping trailing whitespace, retrieves parameters using ``get()`` calls on respective entry fields, and passes them to this router. Exceptions raised by invalid parameters (e.g. Missing keys in Vigenère) are caught via a try-except block and printed directly to the output widget gracefully.

TESTING APPROACH & CHALLENGES

APPROACH

The system was verified using rigorous manual unit tests focusing on standard boundaries and edge cases:

Test Case Description	Input Data	Expected Output	Status
Caesar: Standard text, Shift 3	HELLO WORLD	KHOOR ZRUOG	Pass
Morse: Numeric & Symbols	TEST 123?	- . . . - / .---- ..--- ...-- ..--..	Pass
Vigenere: Missing Key Exception	ATTACK DAWN	Error: A key is required...	Pass
Scytale: Decode Reversibility	HLEL OOWR LD	HELLO WORLD	Pass

CHALLENGES

1. Custom UI Scrollbar Math

Standard tkinter components do not support rounded scrollbars natively; Constructing a native subclassed Canvas object to monitor mouse-wheel deltas and map them to the bounding box ``self.first`` , ``self.last`` .

2. Binary Formatting a Baconi

The Baconian cipher requires exact 5-bit block representation. Mapping Python's default binary strings directly resulted in length truncation for lower alphabetical characters (e.g., 'A' becoming '0' instead of '00000'). This was resolved using the explicit `format(i, '05b')` function.

3. Handling Case Preservation & Symbols

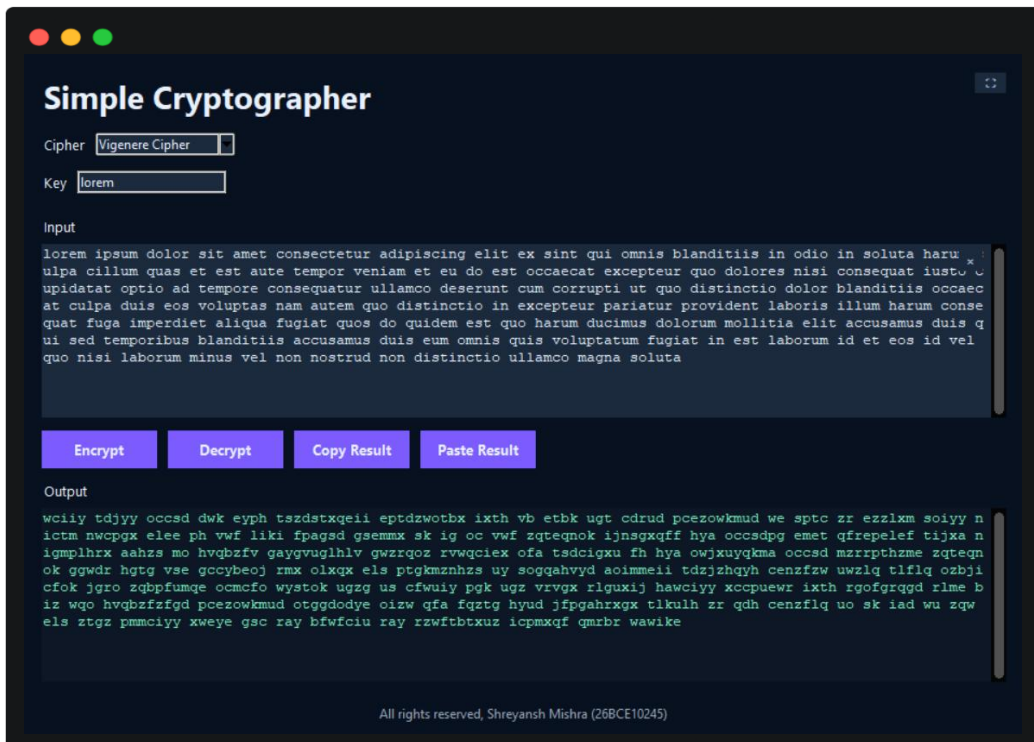
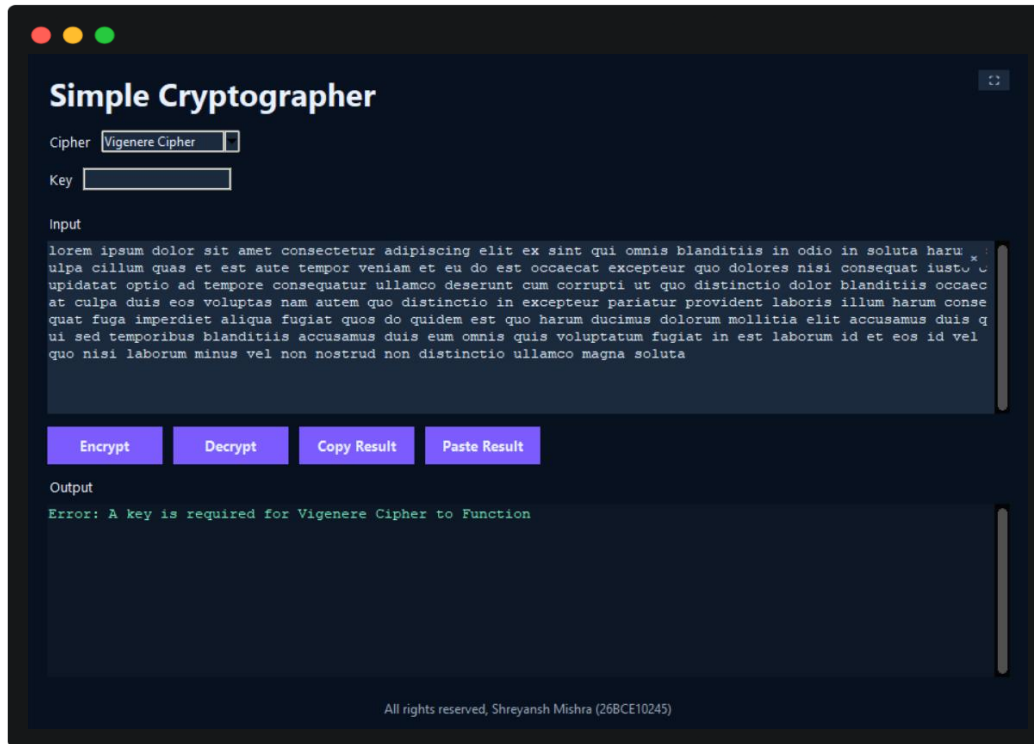
Ensuring that symbols and whitespaces remain unaffected during arithmetic shifts (like in Beaufort and Vigenère) while preserving the original casing of the text meant calculating the ASCII offset using inline ternary operators (`ord('A') if char.isupper() else ord('a')`) on every iteration.

4. State Management in UI

Dynamically showing and hiding the configuration inputs (Shift, Key, Diameter) based on the current ``ComboBox`` selection without messing up the pack/grid layout required utilizing ``grid_remove()`` over ``grid_forget()`` to preserve their original coordinate allocations.

SCREENSHOTS AND RESULTS

Below is a rendered mockup of the primary graphical interface operating in Caesar Cipher mode, illustrating the custom theme and layout configuration.



CONCLUSION

LEARNINGS

This is the second time I have implemented object-oriented programming, and I genuinely think that I can do much better than this after learning the math (analogy) behind it. I've learned 10 historical and classical ciphers and learned the conversion logic of the specifics. Through this project, we've also learned how we can use tkinter to create beautiful GUI apps in python. We've also learned how to create a executable python file.

FUTURE ENHANCEMENTS

Due to the object-oriented implementation of the code, we can dramatically introduce new ciphers like hill cipher or rail fence cipher, even the newer RSH cipher. The GUI could be much better, and we can also look for a host for this program

REFERENCES

1. The Algorithms
<https://github.com/thealgorithms>
2. Tkinter Documentation and Implementation Guide
<https://www.geeksforgeeks.org/python/python-gui-tkinter/>
3. Caesar Cipher
<https://www.geeksforgeeks.org/ethical-hacking/caesar-cipher-in-cryptography/>
4. A1Z26 Cipher
<https://www.catencode.com/ciphers/a1z26>
5. Atbash Cipher
<https://rumkin.com/tools/cipher/atbash/>
6. Baconian Cipher
<https://www.geeksforgeeks.org/python/baconian-cipher/>
7. Beaufort Cipher
<http://practicalcryptography.com/ciphers/beaufort-cipher/>
8. Vigenere Cipher
<https://www.futurelearn.com/info/courses/the-mathematics-of-cryptography-from-ancient-rome-to-a-quantum-future/o/steps/379066>
9. Xor Cipher
<https://www.geeksforgeeks.org/dsa/xor-cipher/>
10. Scytale Cipher
<https://medium.com/@benjamin.calvin/encryption-ciphers-scytale-cipher-e879ebd90c79>
11. Rot13
<https://www.geeksforgeeks.org/dsa/rot13-cipher/>
12. Morse Code
<https://www.geeksforgeeks.org/python/morse-code-translator-python/>

END

A PROJECT FILE BY
Shreyansh Mishra
26BCE10245

September 27, 2026

—

Python Essentials

—

Dr. Lakshmi D.
Dr. Poonkuntran S.
